Koneru Lakshmaiah Education Foundation
(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)
Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.
Phone No: 7815926816, www.klh.edu.in

Department of Computer Science and Engineering
2026-2027
Odd Semester

DATA STRUCTURES AND ALGORITHMS-3 (25CS2103E)

Develop TextHack+: An Intelligent Language Query System Using Custom-Built Data Structures and Algorithms

Jakke Vivek	**2520030180**
Kummari Partheev Praneeth	**2520030197**
Kari Prudhvi Sai	**2520030286**

Team Number: 07

COURSE INSTRUCTOR

Ms. D. Swathi
Assistant Professor
Department of Computer Science and Engineering

TITLE OF THE PBL

Develop TextHack+: An Intelligent Language Query System Using Custom-Built Data Structures and Algorithms (Exact Matching, Fuzzy Search, Document Similarity, Citation-Flow Analysis, Task Scheduling and Primality Testing)

1. CASE STUDY / PROJECT DESCRIPTION

Text analysis is the task of extracting useful answers from large collections of text: finding where a word occurs, tolerating spelling mistakes, deciding how similar two documents are, tracing how influence flows through a network of citations, and ordering batches of processing jobs. These operations sit behind search engines, spell-checkers, plagiarism detectors, digital libraries and clinical-text systems. Most programs solve them by calling ready-made collections and library routines, which hides how the underlying data structures behave and what each algorithm really costs.

TextHack+ is a Java text analysis engine that answers six families of queries through a single layered system: **exact pattern matching** (Knuth-Morris-Pratt), **fuzzy search** (Levenshtein edit distance), **document similarity** (suffix array with an LCP array, giving the longest common substring), **citation-flow analysis** (Edmonds-Karp maximum flow), **task scheduling** (a greedy priority scheduler on a max-heap) and **probabilistic primality testing** (Miller-Rabin). The architecture has four layers: corpus ingestion, a hand-built data structure and algorithm engine, a query service, and a user interface.

The defining constraint of the project is that the engine is built **from scratch, without the java.util collections or algorithm classes**. The dynamic array, linked list, hash map, queue, stack, max-heap, adjacency-list graph and trie are all written by the team. The only library classes used are java.io and java.nio for reading corpus files. Every query returns its answer together with the algorithm name, its theoretical complexity, the measured running time and an operation count, so the tool doubles as an educational and benchmarking aid. The system ships with English, Hindi and medical sample corpora, which demonstrates multi-lingual and domain-specific querying.

1.1 Existing System

- Conventional text tools offer a single function, such as a keyword find, and do not bring exact matching, fuzzy matching, similarity, flow analysis, scheduling and number-theoretic tests into one system.
- Applications normally depend on built-in collections (ArrayList, HashMap, PriorityQueue and similar). These abstract away memory management and the cost of each operation, so the trade-offs stay invisible to the learner.
- Results are usually reported without the algorithm used, its complexity, or any runtime or operation metrics, so the output cannot be used for benchmarking or teaching.
- Support for Indian-language text and domain-specific (for example medical) corpora is often an afterthought.

1.2 Proposed System

- A single Java engine that routes each query type to a dedicated, hand-written algorithm through a unified *QueryService* dispatcher.
- All data structures (DynamicArray, SinglyLinkedList, CustomHashMap, Queue, Stack, MaxHeap, AdjListGraph, Trie) are implemented manually; no java.util collection classes appear in the source.

- Every result is wrapped in a uniform *QueryResult* carrying the answer, algorithm name, Big-O complexity, execution time in nanoseconds and an operation count.
- A menu-driven command-line interface exposes all six query types, with a *Domain Mode* that switches to the medical corpus. A Spring Boot web interface with a REST API exposes exact, fuzzy, similarity and primality queries.
- Nine test harnesses with 86 assertion statements verify each component, from the data-structure layer to the dispatcher.

1.3 Applications / Case Study Examples

Search and retrieval. Exact matching finds every place a keyword appears in a document, as in a browser or editor Find command and in digital libraries. The KMP algorithm guarantees linear time even when the text is highly repetitive.

Spell-checking and fault-tolerant queries. Edit distance measures how many insertions, deletions and substitutions separate two words. A vocabulary spell-checker ranks dictionary words by this distance, and a threshold lets the computation stop early once a word is clearly too far away.

Plagiarism and similarity analysis. The longest common substring of two documents, found with a suffix array and LCP array, is a direct indicator of copied passages. The project compares the English and medical corpora as a demonstration.

Citation-flow analysis. A set of papers and the citations between them forms a directed network. The maximum flow between two papers shows how much influence can pass from one to the other and where the bottleneck edges are.

Batch scheduling and hashing. Text-mining jobs of differing urgency are executed in priority order using a heap. Miller-Rabin primality testing is used to choose a prime bucket count for the project's own hash table, which keeps the load factor low.

Multi-lingual and medical text. The corpora include English, Hindi (Devanagari script) and clinical sentences, illustrating that the same engine works across languages and domains.

1.4 Features of the Developed Application

- Six query types behind one dispatcher: exact match, fuzzy match, similarity, citation flow, job scheduling, primality test.
- Menu-driven CLI with corpus selection (English, Hindi) and a Domain Mode that restricts search to the medical corpus.
- Exact search over the active corpus file or over a custom string, reporting total matches, every match index and the first match.
- Fuzzy search as a vocabulary spell-checker across the corpus, or as an edit distance between two custom strings.
- Document similarity between the English and medical corpora, or between two custom texts, returning the longest common substring and its length.
- Citation-flow analysis on a predefined 6-paper network or on a user-supplied capacity matrix.
- Task scheduling on a predefined job set or on custom jobs (priority and duration).
- Primality testing of a single number, and a recommendation for the next prime capacity for the custom hash map.

- Spring Boot web UI with tabs for Exact Match, Fuzzy Search, Similarity and Primality Test, backed by REST endpoints under /api/query.
- Uniform result metadata for every query: algorithm, complexity, runtime and operation count.

1.5 Hardware and Software Requirements

Item	Requirement
Programming language	Java Development Kit (JDK) 8 or higher for the core engine and CLI. The Spring Boot web layer (version 3.1.5, Maven) is configured for Java 17. The project was re-compiled and all tests re-run on JDK 21 while preparing this report.
Core constraint	No java.util collections; custom lists, maps, trees, queues, stacks and heaps (java.io / java.nio used only for file input).
Operating system	Cross-platform: Windows, Linux or macOS.
Datasets	Sample English, Hindi and medical corpora supplied in the repository; Indian-language Wikipedia and medical text datasets as target data.
Processor	Multi-core Intel Core i5/i7 or equivalent.
RAM	8 GB minimum; 16 GB recommended for large datasets.
Storage	Standard SSD/HDD with space for local text datasets.
Tools	Any Java IDE or VS Code with the Java extensions; Git/GitHub for version control (repository: prudhvisaikari-hub/DSA3-PROJECT).

1.6 Problem Statement

Standard high-level frameworks and built-in collections abstract away the internal mechanics of data structures and memory management. When complex queries are run over multi-lingual datasets such as Indian-language Wikipedia, or over domain-specific data, relying on generic libraries obscures the computational trade-offs. The challenge is to engineer a multi-functional text processing engine entirely from scratch, without pre-built collections, while keeping the algorithms efficient across very different query types.

1.7 Objectives

- **Custom data structure implementation:** manually construct all data structures and algorithms in Java without importing java.util collections.
- **Multi-modal query services:** exact pattern matching, fuzzy search with edit distance, document similarity with suffix structures, citation-flow analysis with max-flow, task scheduling illustrations for NP-hard problems, and probabilistic primality testing.
- **Multi-lingual and domain adaptation:** process Indian-language corpora and domain applications such as medical text analysis.
- **Educational and research benchmarking:** display both the computed result and runtime metrics for every algorithm.
- **Validation:** verify every component with automated test harnesses and measure operation counts and run times on controlled inputs.

ALGORITHM / PSEUDO CODE

2. SYSTEM DESIGN AND ALGORITHMS

2.1 Layered Architecture and Flow of the Application

The engine is organised in four layers. Data flows from the corpus files upward to the user interface, and every query passes through the same dispatcher.

Layer	Package	Role
1. Data structures	com.texthackplus.structures	DynamicArray, SinglyLinkedList, CustomHashMap (chaining, load factor 0.7), Queue, Stack, MaxHeap, AdjListGraph, Trie.
2. Ingestion	com.texthackplus.ingestion	CorpusLoader (file I/O), Tokenizer, Document, DocumentStore with a token-to-document posting index.
3. Algorithm engine and service	com.texthackplus.algorithms, .service	KMP, Levenshtein, suffix array + LCP, Edmonds-Karp, job scheduler, Miller-Rabin; QueryService dispatches and QueryResult carries metadata.
4. User interface	.cli, .controller, static web files	Menu-driven CLI; Spring Boot REST controller (/api/query/*) with an HTML/CSS/JavaScript front end.

Flow of a query: (1) the corpus is ingested, (2) the user selects a query or operation type and supplies input (keyword, text pattern, two documents, a capacity matrix, a job list or a number), (3) the dispatcher routes the request to the matching hand-written algorithm, (4) the result is aggregated with runtime and operation metrics, and (5) the answer and the algorithm information are shown to the user.

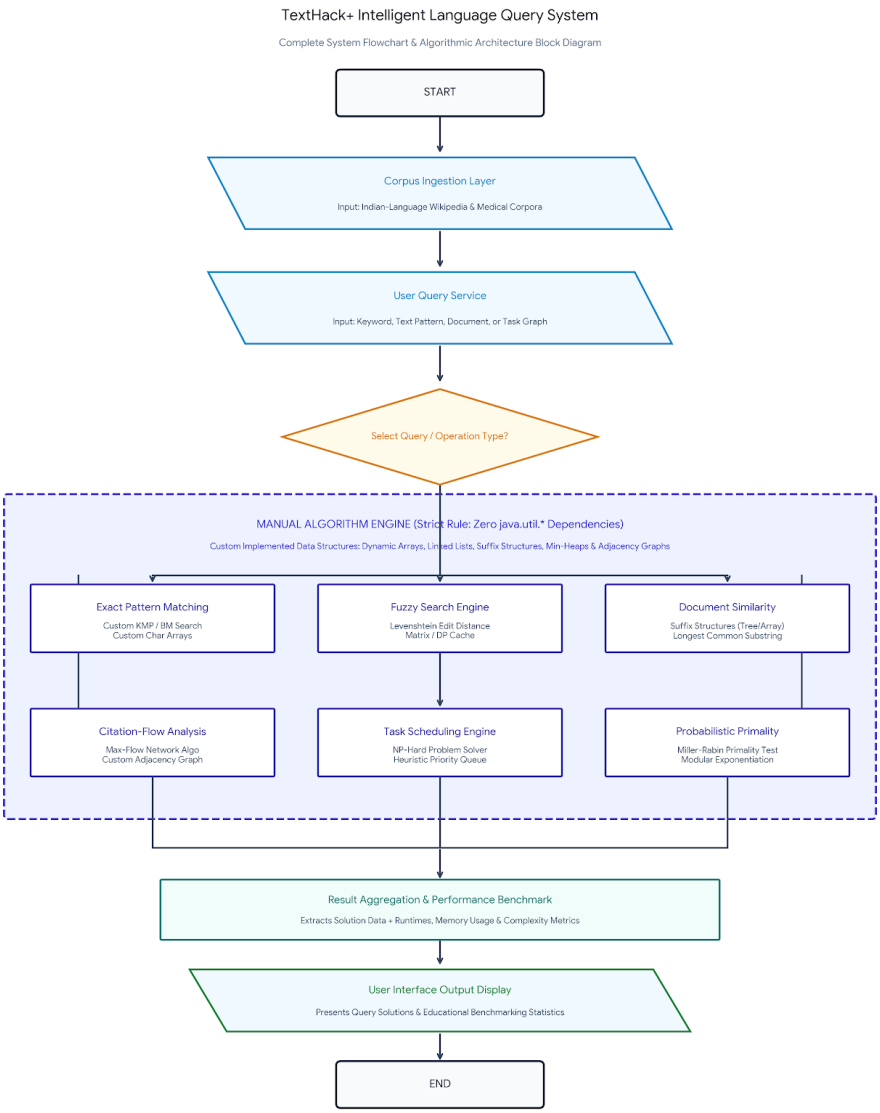


Figure 2.1: System flowchart and algorithmic architecture block diagram (from the project presentation).

2.2 Exact Pattern Matching - Knuth-Morris-Pratt (KMP)

KMP avoids re-comparing text characters after a mismatch. A preprocessing pass builds the *LPS* array (longest proper prefix that is also a suffix) of the pattern. During the search, when a mismatch occurs at pattern index j, the pattern pointer falls back to LPS[j-1] instead of restarting, and the text pointer never moves backwards. This gives O(n + m) time for a text of length n and a pattern of length m. The project provides *search* (first match, with a comparison counter) and *searchAll* (every match index, stored in the custom DynamicArray).

```
BUILD-LPS(P)
  m <- length(P);  lps[0..m-1] <- 0;  len <- 0;  i <- 1
  while i < m
    if P[i] = P[len]            lps[i] <- ++len;  i <- i + 1
    else if len != 0            len <- lps[len - 1]
    else                        lps[i] <- 0;  i <- i + 1
  return lps

KMP-SEARCH-ALL(T, P)
  if m = 0 or n = 0 return empty list
  lps <- BUILD-LPS(P);  i <- 0;  j <- 0
  while i < n
    if T[i] = P[j]
      i <- i + 1;  j <- j + 1
      if j = m   add (i - j) to matches;  j <- lps[j - 1]
    else if j != 0   j <- lps[j - 1]
    else             i <- i + 1
  return matches
```

2.3 Fuzzy Search - Levenshtein Edit Distance

The Levenshtein distance is the minimum number of single-character insertions, deletions and substitutions needed to turn one string into another. It is computed with dynamic programming. The implementation keeps only two rows of the table (so memory is O(min(n, m))), always treats the shorter string as the column dimension, and supports an optional threshold: if the smallest value in a row already exceeds the allowed distance, the computation stops early. For example, the distance between "saturday" and "sunday" is 3.

```
LEVENSHTEIN(s1, s2, maxDistance)
  n <- |s1|, m <- |s2|;  if n = 0 return m;  if m = 0 return n
  if m > n  swap(s1, s2) and swap(n, m)            // keep rows short
  prev[j] <- j for j = 0..m
  for i <- 1 to n
    cur[0] <- i;  minInRow <- cur[0]
    for j <- 1 to m
      cost   <- 0 if s1[i-1] = s2[j-1] else 1
      cur[j] <- min(prev[j] + 1,  cur[j-1] + 1,  prev[j-1] + cost)
      minInRow <- min(minInRow, cur[j])
    if minInRow > maxDistance  return minInRow     // early stop
    swap(prev, cur)
  return prev[m]
```

2.4 Document Similarity - Suffix Array and LCP Array

To measure similarity the system finds the **longest common substring** of two documents A and B. The two texts are joined as S = A + "#" + B. A suffix array of S lists all suffix start positions in sorted order (the project sorts them with its own quicksort over suffix comparisons). Kasai's algorithm then builds the LCP array in O(n): LCP[i] is the length of the common prefix of the suffixes at ranks i-1 and i. The longest common substring is the largest LCP value between two adjacent suffixes that originate from different documents. Because suffix comparison can cost O(n), the whole procedure is O(n^2 log n) in the worst case, as stated by the implementation.

```
LONGEST-COMMON-SUBSTRING(A, B)
  S <- A + '#' + B;  sep <- |A|;  n <- |S|
  SA  <- sort suffix indices 0..n-1 by comparing suffixes of S      // quicksort
  LCP <- KASAI(S, SA)
  maxLen <- 0
  for i <- 1 to n-1
    p1 <- SA[i-1];  p2 <- SA[i]
    if one of p1, p2 lies in A (< sep) and the other in B (> sep)
      if LCP[i] > maxLen   maxLen <- LCP[i];  best <- p1
  return S[best .. best + maxLen)

KASAI(S, SA)
  inv[SA[i]] <- i;  k <- 0
  for i <- 0 to n-1
    if inv[i] > 0
      j <- SA[inv[i] - 1]
      while S[i+k] = S[j+k]   k <- k + 1
      LCP[inv[i]] <- k;  if k > 0  k <- k - 1
```

2.5 Citation-Flow Analysis - Edmonds-Karp Maximum Flow

A citation network is modelled as a directed graph whose edge capacities describe how strongly one paper feeds another. Edmonds-Karp is the Ford-Fulkerson method with breadth-first search used to pick the shortest augmenting path each time. The BFS uses the project's own Queue class. The search repeats until no augmenting path remains, and the sum of the bottleneck values is the maximum flow. Its running time is O(V * E^2).

```
EDMONDS-KARP(cap, source, sink)
  residual <- copy of cap;  maxFlow <- 0
  while BFS(residual, source, sink, parent)         // shortest augmenting path
    pathFlow <- min residual[parent[v]][v] along the path sink -> source
    for each edge (u, v) on the path
      residual[u][v] <- residual[u][v] - pathFlow
      residual[v][u] <- residual[v][u] + pathFlow
    maxFlow <- maxFlow + pathFlow
  return maxFlow
```

2.6 Task Scheduling - Greedy Priority Scheduler

The scheduling demonstration models a batch processor for text-mining jobs. Each job has an id, a priority and a duration. All jobs are inserted into the custom MaxHeap and then extracted one by one. The ordering rule is: higher priority first; when priorities are equal, the shorter duration goes first (shortest-job-first). This greedy heuristic runs in O(n log n) and illustrates how heuristic strategies are used for scheduling problems that are NP-hard in their general form.

```
SCHEDULE(jobs)
  heap <- empty MaxHeap          // order: priority desc, then duration asc
  for each job in jobs   heap.insert(job)
  order <- empty list
  while heap is not empty   order.add( heap.extractMax().id )
  return order
```

2.7 Probabilistic Primality Testing - Miller-Rabin

Miller-Rabin tests whether an odd number n is probably prime. Write n - 1 = 2^s * d with d odd. For a random base a, compute a^d mod n by fast modular exponentiation; n passes the round if the result is 1 or n - 1, or if repeated squaring reaches n - 1. A composite number survives a round only rarely, so k independent rounds make a wrong 'prime' answer extremely unlikely, while a composite is rejected as soon as one round fails. The implementation avoids java.util.Random by drawing bases from a small xor-shift generator, and uses a double-and-add multiplication to avoid overflow. The same routine is used to find the next prime bucket count for the project's hash table.

```
IS-PRIME(n, k)
  if n <= 1 return false;  if n <= 3 return true;  if n is even return false
  d <- n - 1;  while d is even  d <- d / 2
  repeat k times
    a <- pseudo-random value in [2, n - 2]
    if not MILLER-TEST(d, n, a)  return false          // definitely composite
  return true                                           // probably prime

MILLER-TEST(d, n, a)
  x <- POWER(a, d, n)
  if x = 1 or x = n - 1  return true
  while d != n - 1
    x <- x * x mod n;  d <- 2d
    if x = 1 return false;  if x = n - 1 return true
  return false
```

2.8 Complexity Analysis

Query	Algorithm	Time complexity	Extra space
Exact match	KMP	O(n + m)	O(m) LPS array
Fuzzy match	Levenshtein (two-row DP)	O(n * m)	O(min(n, m))
Document similarity	Suffix array + Kasai LCP	O(n^2 log n) as implemented; LCP in O(n)	O(n)
Citation flow	Edmonds-Karp	O(V * E^2)	O(V^2) residual matrix
Task scheduling	Greedy priority on MaxHeap	O(n log n)	O(n) heap
Primality test	Miller-Rabin	O(k log^3 n) (as stated in code)	O(1)

Table 2.1: Complexity of the six query algorithms (n, m are input lengths, V and E graph size, k number of rounds).

CODE

3. PROJECT STRUCTURE

Package / File	Purpose
structures/*	DynamicArray, SinglyLinkedList, CustomHashMap, Queue, Stack, MaxHeap, AdjListGraph, Trie: the hand-written data structures.
ingestion/*	CorpusLoader, Tokenizer, Document, DocumentStore: load, tokenise and index the English, Hindi and medical corpora.
algorithms/exact/KMPMatcher.java	Exact pattern matching in O(n + m), first match and all matches.
algorithms/fuzzy/Levenshtein.java	Edit distance by dynamic programming with optional early stop.
algorithms/similarity/SuffixTreeSimilarity.java	Suffix array, Kasai LCP and longest common substring.
algorithms/flow/MaxFlowCitation.java	Edmonds-Karp maximum flow on a capacity matrix.
algorithms/scheduling/JobScheduler.java	Priority job scheduler using the custom MaxHeap.
algorithms/math/MillerRabinPrimality.java	Probabilistic primality test.
service/QueryService.java, QueryResult.java	Unified dispatcher and uniform result object (answer, algorithm, complexity, time, operations).
cli/CLI.java	Menu-driven command-line interface (corpus selection, Domain Mode, six queries).
controller/QueryController.java, resources/static/*	Spring Boot REST endpoints and the HTML/CSS/JS web front end.
src/test/java/*	Nine assertion-based test harnesses, one per component.

Table 3.1: Source organisation of the repository.

The listings below are reproduced directly from the project repository. Result-holder boilerplate (the nested SearchResult classes with getters) is omitted for brevity.

3.1 QueryService.java (dispatcher, excerpt)

```
public enum QueryType {
    EXACT_MATCH,
    FUZZY_MATCH,
    SIMILARITY,
    CITATION_FLOW,
    JOB_SCHEDULING,
    PRIMALITY_TEST
}

public static QueryResult executeQuery(QueryType type, Object... params) {
    switch (type) {
            case EXACT_MATCH: {
                // params: String text, String pattern
                String text = (String) params[0];
                String pattern = (String) params[1];
                KMPMatcher.SearchResult res = KMPMatcher.search(text, pattern);
                return new QueryResult(
                        res.getIndex(),
                        res.getAlgorithm(),
                        res.getComplexity(),
                        res.getTimeNs(),
                        res.getComparisons()
                );
            }
        // ... FUZZY_MATCH, SIMILARITY, CITATION_FLOW, JOB_SCHEDULING,
        //     PRIMALITY_TEST follow the same pattern ...
    }
}
```

3.2 KMPMatcher.java - searchAll

```
public static com.texthackplus.structures.array.DynamicArray<Integer> searchAll(String text, String pattern) {
    com.texthackplus.structures.array.DynamicArray<Integer> matches = new com.texthackplus.structures.array.DynamicArr
      ay<>();
    int n = text.length();
    int m = pattern.length();
    if (m == 0 || n == 0) return matches;

    int[] lps = new int[m];
    int len = 0;
    int i = 1;
    while (i < m) {
        if (pattern.charAt(i) == pattern.charAt(len)) {
            len++;
            lps[i] = len;
            i++;
        } else {
            if (len != 0) {
                len = lps[len - 1];
            } else {
                lps[i] = 0;
                i++;
            }
        }
    }

    i = 0;
    int j = 0;
    while (i < n) {
        if (text.charAt(i) == pattern.charAt(j)) {
            i++;
            j++;
            if (j == m) {
                matches.add(i - j);
                j = lps[j - 1];
            }
        } else {
            if (j != 0) {
                j = lps[j - 1];
            } else {
                i++;
            }
        }
    }
    return matches;
}
```

3.3 Levenshtein.java - computeDistance

```
public static SearchResult computeDistance(String s1, String s2, int maxDistance) {
    long start = System.nanoTime();
    int comparisons = 0;
    int n = s1.length();
    int m = s2.length();

    // Edge cases - one string empty.
    if (n == 0) {
        long end = System.nanoTime();
        return new SearchResult(m, comparisons, end - start);
    }
    if (m == 0) {
        long end = System.nanoTime();
        return new SearchResult(n, comparisons, end - start);
    }

    // Use only two rows to keep memory O(min(n,m)). Ensure m is the shorter string.
    if (m > n) {
        // swap to make m <= n for less memory.
        String tmp = s1; s1 = s2; s2 = tmp;
        int tmpLen = n; n = m; m = tmpLen;
    }

    int[] prev = new int[m + 1]; // previous row of DP table
    int[] cur = new int[m + 1];  // current row

    // Initialize base case: distance from empty prefix of s1 to prefixes of s2.
    for (int j = 0; j <= m; j++) {
        prev[j] = j;
    }

    for (int i = 1; i <= n; i++) {
        cur[0] = i;
        int minInRow = cur[0];
        char c1 = s1.charAt(i - 1);
        for (int j = 1; j <= m; j++) {
            comparisons++;
            char c2 = s2.charAt(j - 1);
            int cost = (c1 == c2) ? 0 : 1;
            int del = prev[j] + 1;      // deletion
            int ins = cur[j - 1] + 1;   // insertion
            int sub = prev[j - 1] + cost; // substitution
            int best = del;
            if (ins < best) best = ins;
            if (sub < best) best = sub;
            cur[j] = best;
            if (best < minInRow) minInRow = best;
        }
        // Early termination check - if the minimal value of the current row
        // already exceeds the allowed threshold, we can stop.
        if (minInRow > maxDistance) {
            long end = System.nanoTime();
            return new SearchResult(minInRow, comparisons, end - start);
        }
        // swap rows for next iteration
        int[] tmpRow = prev; prev = cur; cur = tmpRow;
    }
    long end = System.nanoTime();
    return new SearchResult(prev[m], comparisons, end - start);
}
```

3.4 SuffixTreeSimilarity.java - longest common substring and LCP

```
public static SearchResult longestCommonSubstring(String a, String b) {
    long start = System.nanoTime();
    int comparisons = 0;

    if (a.isEmpty() || b.isEmpty()) {
        return new SearchResult("", 0, comparisons, System.nanoTime() - start);
    }

    // 1. Combine strings with a unique separator
    String s = a + "#" + b;
    int sepIdx = a.length();
    int n = s.length();

    // 2. Build Suffix Array
    int[] sa = buildSuffixArray(s);

    // 3. Build LCP Array (Kasai's algorithm)
    int[] lcp = buildLCPArray(s, sa);

    // 4. Find the max LCP between adjacent suffixes from different source strings
    int maxLen = 0;
    int bestStart = -1;

    for (int i = 1; i < n; i++) {
        comparisons++;
        int pos1 = sa[i - 1];
        int pos2 = sa[i];

        // Check if one is from 'a' (before '#') and one is from 'b' (after '#')
        boolean p1InA = pos1 < sepIdx;
        boolean p2InA = pos2 < sepIdx;
        boolean p1InB = pos1 > sepIdx;
        boolean p2InB = pos2 > sepIdx;

        if ((p1InA && p2InB) || (p1InB && p2InA)) {
            if (lcp[i] > maxLen) {
                maxLen = lcp[i];
                bestStart = pos1; // Does not matter which pos we take, they match up to maxLen
            }
        }
    }

    String best = maxLen > 0 ? s.substring(bestStart, bestStart + maxLen) : "";
    long end = System.nanoTime();
    return new SearchResult(best, maxLen, comparisons, end - start);
}

public static int[] buildLCPArray(String s, int[] sa) {
    int n = s.length();
    int[] lcp = new int[n];
    int[] inv = new int[n];
    for (int i = 0; i < n; i++) {
        inv[sa[i]] = i;
    }

    int k = 0;
    for (int i = 0; i < n; i++) {
        if (inv[i] > 0) {
            int j = sa[inv[i] - 1];
            while (i + k < n && j + k < n && s.charAt(i + k) == s.charAt(j + k)) {
                k++;
            }
            lcp[inv[i]] = k;
            if (k > 0) {
                k--;
            }
        }
    }
    return lcp;
}
```

3.5 MaxFlowCitation.java - Edmonds-Karp

```
public static SearchResult computeMaxFlow(int[][] capacities, int source, int sink) {
    long start = System.nanoTime();
    int comparisons = 0;
    int n = capacities.length;

    // Create a residual capacity matrix.
    int[][] residual = new int[n][n];
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            residual[i][j] = capacities[i][j];
        }
    }

    int[] parent = new int[n];
    int maxFlow = 0;

    // While there is an augmenting path from source to sink...
    while (bfs(residual, source, sink, parent)) {
        comparisons++; // One augmenting path found

        // Find the bottleneck capacity in the augmenting path.
        int pathFlow = Integer.MAX_VALUE;
        for (int v = sink; v != source; v = parent[v]) {
            int u = parent[v];
            if (residual[u][v] < pathFlow) {
                pathFlow = residual[u][v];
            }
        }

        // Update residual capacities along the path.
        for (int v = sink; v != source; v = parent[v]) {
            int u = parent[v];
            residual[u][v] -= pathFlow;
            residual[v][u] += pathFlow;
        }

        maxFlow += pathFlow;
    }

    long end = System.nanoTime();
    return new SearchResult(maxFlow, comparisons, end - start);
}

private static boolean bfs(int[][] residual, int source, int sink, int[] parent) {
    int n = residual.length;
    boolean[] visited = new boolean[n];
    Queue<Integer> queue = new Queue<>();

    queue.enqueue(source);
    visited[source] = true;
    parent[source] = -1;

    while (!queue.isEmpty()) {
        int u = queue.dequeue();

        for (int v = 0; v < n; v++) {
            if (!visited[v] && residual[u][v] > 0) {
                queue.enqueue(v);
                parent[v] = u;
                visited[v] = true;
                if (v == sink) {
                    return true;
                }
            }
        }
    }
    return false;
}
```

3.6 JobScheduler.java - schedule

```
public static SearchResult schedule(Job[] jobs) {
    long start = System.nanoTime();
    int comparisons = 0;

    MaxHeap<Job> heap = new MaxHeap<>();
    for (int i = 0; i < jobs.length; i++) {
        heap.insert(jobs[i]);
        comparisons++; // One insert operation represents some work
    }

    DynamicArray<Integer> order = new DynamicArray<>();
    while (heap.size() > 0) {
        Job job = heap.extractMax();
        order.add(job.getId());
        comparisons++; // One extract operation represents some work
    }

    long end = System.nanoTime();
    return new SearchResult(order, comparisons, end - start);
}
```

3.7 MillerRabinPrimality.java

```
public static SearchResult isPrime(long n, int iterations) {
    long start = System.nanoTime();
    int comparisons = 0;

    if (n <= 1) {
        return new SearchResult(false, comparisons, System.nanoTime() - start);
    }
    if (n <= 3) {
        return new SearchResult(true, comparisons, System.nanoTime() - start);
    }
    if (n % 2 == 0) {
        return new SearchResult(false, comparisons, System.nanoTime() - start);
    }

    long d = n - 1;
    while (d % 2 == 0) {
        d /= 2;
    }

    boolean prime = true;
    for (int i = 0; i < iterations; i++) {
        comparisons++;
        // Generate a pseudo-random base 'a' in the range [2, n - 2]
        // We use a simple linear congruential generator to avoid java.util.Random
        long a = 2 + (Math.abs(pseudoRandom(i)) % (n - 3));

        if (!millerTest(d, n, a)) {
            prime = false;
            break;
        }
    }

    long end = System.nanoTime();
    return new SearchResult(prime, comparisons, end - start);
}

private static boolean millerTest(long d, long n, long a) {
    long x = power(a, d, n);
    if (x == 1 || x == n - 1) {
        return true;
    }

    while (d != n - 1) {
        x = mulMod(x, x, n);
        d *= 2;

        if (x == 1) return false;
        if (x == n - 1) return true;
    }
    return false;
}

private static long power(long base, long exp, long mod) {
```

```
        long res = 1;
        base = base % mod;
        while (exp > 0) {
            if ((exp & 1) == 1) {
                res = mulMod(res, base, mod);
            }
            exp = exp >> 1;
            base = mulMod(base, base, mod);
        }
        return res;
    }

    private static long mulMod(long a, long b, long mod) {
        long res = 0;
        a %= mod;
        while (b > 0) {
            if ((b & 1) == 1) {
                res = (res + a) % mod;
            }
            a = (a * 2) % mod;
            b >>= 1;
        }
        return res;
    }
```

3.8 QueryController.java (REST layer)

```
package com.texthackplus.controller;


@RestController
@RequestMapping("/api/query")
public class QueryController {

    static class MatchRequest {
        public String text1;
        public String text2;
    }

    static class PrimalityRequest {
        public long number;
    }

    @PostMapping("/exact")
    public QueryResult exactMatch(@RequestBody MatchRequest req) {
        return QueryService.executeQuery(QueryType.EXACT_MATCH, req.text1, req.text2);
    }

    @PostMapping("/fuzzy")
    public QueryResult fuzzyMatch(@RequestBody MatchRequest req) {
        return QueryService.executeQuery(QueryType.FUZZY_MATCH, req.text1, req.text2);
    }

    @PostMapping("/similarity")
    public QueryResult similarity(@RequestBody MatchRequest req) {
        return QueryService.executeQuery(QueryType.SIMILARITY, req.text1, req.text2);
    }

    @PostMapping("/primality")
    public QueryResult primality(@RequestBody PrimalityRequest req) {
        return QueryService.executeQuery(QueryType.PRIMALITY_TEST, req.number, 5);
    }
}
```

3.9 Building and Running

The core engine has no external dependencies, so it can be compiled and run with the JDK alone. The README gives the following steps for Windows; the same commands work on other systems with the usual path syntax.

```
mkdir bin
dir /b /s src\main\java\*.java src\test\java\*.java > allsources.txt
javac -d bin @allsources.txt
java -cp bin com.texthackplus.cli.CLI                        (run the CLI)
java -ea -cp bin com.texthackplus.service.QueryServiceTest  (run a test harness; -ea enables assertions)
```

Note: on a plain JDK, compile only the non-Spring sources (everything except controller/ and TextHackPlusApplication.java). To run the web interface, use Maven with the supplied pom.xml (Spring Boot 3.1.5, Java 17), for example `mvn spring-boot:run`, and open the application in a browser; the page offers tabs for Exact Match, Fuzzy Search, Similarity and Primality Test.

3.10 User Interface Behaviour

- **CLI:** the main menu lists Select Corpus, Toggle Domain Mode, Exact Match (KMP), Fuzzy Match (Levenshtein), Document Similarity (suffix array + LCP), Citation-Flow Analysis (Edmonds-Karp), Task Scheduling Demo (priority queue / MaxHeap), Primality Test and Hash Sizing (Miller-Rabin), and Exit. Each result is printed with the answer, algorithm, complexity, time in milliseconds and operations.
- **Domain Mode:** when ON, only the medical corpus (medical.txt) is active; when OFF the general English corpus is used.
- **Web UI:** one tab per query type. The front end posts JSON to /api/query/exact, /fuzzy, /similarity or /primality and displays the returned result box with its metadata. The exact-match endpoint reports the first match index.

RESULTS

4.1 APPLICATION OUTPUT (CLI)

The following screenshots, taken from the team's runs of the CLI, show the main menu with corpus selection and each of the six query types. Every result is printed with the algorithm used, its complexity, the runtime and the operation count.

```
PS C:\texthack-plus>  & 'C:\Users\prudh\.antigravity-ide\extensions\redhat.java-1.56.0-win32-x64\jre\21.0.12.1-win32-x86_64\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\texthack-plus\target\classes' 'com.team7.texthackplus.cli.CLI'
====================================
   Welcome to TextHack+ Engine CLI
====================================
Current Corpus: english.txt | Domain Mode: OFF (General)
1. Select Corpus
2. Toggle Domain Mode
3. Exact Match (KMP Algorithm on Corpus / Text)
4. Fuzzy Match (Levenshtein Spell-Checker / Distance)
5. Document Similarity (Suffix Array + LCP Plagiarism Analysis)
6. Citation-Flow Analysis (Edmonds-Karp Max Flow)
7. Task Scheduling Demo (Priority Queue / MaxHeap)
8. Primality Test & Hash Sizing (Miller-Rabin)
9. Exit
Enter your choice: 1
Select a corpus:
1. English (english.txt)
2. Hindi (hindi.txt)
Choice: 1
Selected english.txt

Current Corpus: english.txt | Domain Mode: OFF (General)
1. Select Corpus
2. Toggle Domain Mode
3. Exact Match (KMP Algorithm on Corpus / Text)
4. Fuzzy Match (Levenshtein Spell-Checker / Distance)
5. Document Similarity (Suffix Array + LCP Plagiarism Analysis)
6. Citation-Flow Analysis (Edmonds-Karp Max Flow)
7. Task Scheduling Demo (Priority Queue / MaxHeap)
8. Primality Test & Hash Sizing (Miller-Rabin)
9. Exit
Enter your choice:
```

Figure 4.1: Welcome menu of the TextHack+ CLI and corpus selection (English or Hindi).

```
PS C:\texthack-plus>  c:; cd 'c:\texthack-plus'; & 'C:\Users\prudh\.antigravity-ide\extensions\redhat.java-1.56.0-win32-x64\jre\21.0.12.1-win32-x86_64\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\texthack-plus\target\classes' 'com.team7.texthackplus.cli.CLI'
1. Select Corpus
2. Toggle Domain Mode
3. Exact Match (KMP Algorithm on Corpus / Text)
4. Fuzzy Match (Levenshtein Spell-Checker / Distance)
5. Document Similarity (Suffix Array + LCP Plagiarism Analysis)
6. Citation-Flow Analysis (Edmonds-Karp Max Flow)
7. Task Scheduling Demo (Priority Queue / MaxHeap)
8. Primality Test & Hash Sizing (Miller-Rabin)
9. Exit
Enter your choice: 3
--- Exact Match (KMP) ---
1. Search pattern across active Corpus file (english.txt)
2. Search pattern in custom string input
Choice [1/2]: 2
Enter text to search in: the for is angry
Enter pattern to search for: for
---- KMP Analysis Result ----
Total Matches Found: 1
Match Indices: 4
First Match Index:   4
Algorithm:           KMP
Complexity:          O(n + m)
Time (ms):           0
Operations:          9
----------------------------
```

Figure 4.2: Exact match (KMP). Pattern "for" in "the for is angry": 1 match at index 4, 9 operations.

```
PS C:\texthack-plus>  & 'C:\Users\prudh\.antigravity-ide\extensions\redhat.java-1.56.0-win32-x64\jre\21.0.12.1-win32-x86_64\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\texthack-plus\target\classes' 'com.team7.texthackplus.cli.CLI'
Current Corpus: english.txt | Domain Mode: OFF (General)
1. Select Corpus
2. Toggle Domain Mode
3. Exact Match (KMP Algorithm on Corpus / Text)
4. Fuzzy Match (Levenshtein Spell-Checker / Distance)
5. Document Similarity (Suffix Array + LCP Plagiarism Analysis)
6. Citation-Flow Analysis (Edmonds-Karp Max Flow)
7. Task Scheduling Demo (Priority Queue / MaxHeap)
8. Primality Test & Hash Sizing (Miller-Rabin)
9. Exit
Enter your choice: 4
--- Fuzzy Match (Levenshtein Distance) ---
1. Vocabulary Spell-Checker across Corpus
2. Compute Edit Distance between two custom strings
Choice [1/2]: 2
Enter first string: saturday
Enter second string: sunday
---- Result ----
Answer:       3
Algorithm:    Levenshtein
Complexity:   O(n·m)
Time (ms):    0
Operations:   48
----------------
```

Figure 4.3: Fuzzy match (Levenshtein). "saturday" vs "sunday": distance 3 in 48 operations.

```
PS C:\texthack-plus>  c:; cd 'c:\texthack-plus'; & 'C:\Users\prudh\.antigravity-ide\extensions\redhat.java-1.56.0-win32-x64\jre\21.0.12.1-win32-x86_64\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\texthack-plus\target\classes' 'com.team7.texthackplus.cli.CLI'
Current Corpus: english.txt | Domain Mode: OFF (General)
1. Select Corpus
2. Toggle Domain Mode
3. Exact Match (KMP Algorithm on Corpus / Text)
4. Fuzzy Match (Levenshtein Spell-Checker / Distance)
5. Document Similarity (Suffix Array + LCP Plagiarism Analysis)
6. Citation-Flow Analysis (Edmonds-Karp Max Flow)
7. Task Scheduling Demo (Priority Queue / MaxHeap)
8. Primality Test & Hash Sizing (Miller-Rabin)
9. Exit
Enter your choice: 5
--- Document Similarity (Suffix Array + LCP) ---
1. Compare Corpus Files (english.txt vs medical.txt)
2. Compare two custom text strings
Choice [1/2]: 1
Comparing 'english.txt' (166 chars) vs 'medical.txt' (149 chars)...
---- Document Similarity Analysis ----
Longest Common Substring: "s and "
Common Substring Length:  6
Algorithm:                Suffix Array + LCP
Complexity:               O(n^2 log n)
Time (ms):                0
Operations:               315
-------------------------------------
```

Figure 4.4: Document similarity (suffix array + LCP). english.txt vs medical.txt: longest common substring "s and " (length 6), 315 operations.

```
PS C:\texthack-plus>  & 'C:\Users\prudh\.antigravity-ide\extensions\redhat.java-1.56.0-win32-x64\jre\21.0.12.1-win32-x86_64\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\texthack-plus\target\classes' 'com.team7.texthackplus.cli.CLI'
====================================
Current Corpus: english.txt | Domain Mode: OFF (General)
1. Select Corpus
2. Toggle Domain Mode
3. Exact Match (KMP Algorithm on Corpus / Text)
4. Fuzzy Match (Levenshtein Spell-Checker / Distance)
5. Document Similarity (Suffix Array + LCP Plagiarism Analysis)
6. Citation-Flow Analysis (Edmonds-Karp Max Flow)
7. Task Scheduling Demo (Priority Queue / MaxHeap)
8. Primality Test & Hash Sizing (Miller-Rabin)
9. Exit
Enter your choice: 6
--- Citation-Flow Analysis (Edmonds-Karp Max Flow) ---
1. Predefined 6-Node Paper Citation Network
2. Custom Citation Network Matrix Input
Choice [1/2]: 1
Using predefined paper citation network (6 papers, Paper 0 -> Paper 5).
---- Result ----
Answer:       23
Algorithm:    Edmonds-Karp Max Flow
Complexity:   O(V * E^2)
Time (ms):    9
Operations:   3
----------------
```

Figure 4.5: Citation-flow analysis (Edmonds-Karp) on the predefined 6-paper network: maximum flow 23.

```
PS C:\texthack-plus>  & 'C:\Users\prudh\.antigravity-ide\extensions\redhat.java-1.56.0-win32-x64\jre\21.0.12.1-win32-x86_64\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\texthack-plus\target\classes' 'com.team7.texthackplus.cli.CLI'
====================================
   Welcome to TextHack+ Engine CLI
====================================
Current Corpus: english.txt | Domain Mode: OFF (General)
1. Select Corpus
2. Toggle Domain Mode
3. Exact Match (KMP Algorithm on Corpus / Text)
4. Fuzzy Match (Levenshtein Spell-Checker / Distance)
5. Document Similarity (Suffix Array + LCP Plagiarism Analysis)
6. Citation-Flow Analysis (Edmonds-Karp Max Flow)
7. Task Scheduling Demo (Priority Queue / MaxHeap)
8. Primality Test & Hash Sizing (Miller-Rabin)
9. Exit
Enter your choice: 7
--- Task Scheduling Engine (MaxHeap Priority Queue) ---
1. Default Predefined Job Set
2. Enter Custom Job Queue
Choice [1/2]: 2
Enter number of batch jobs to schedule: 6
Job #1:
  Priority (higher = more urgent): 2
  Duration (ms): 3
Job #2:
  Priority (higher = more urgent): 9
  Duration (ms): 3
Job #3:
  Priority (higher = more urgent): 5
  Duration (ms): 3
```

Figure 4.6: Task scheduling demo with custom job input (priority and duration per job).

```
PS C:\texthack-plus>  & 'C:\Users\prudh\.antigravity-ide\extensions\redhat.java-1.56.0-win32-x64\jre\21.0.12.1-win32-x86_64\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\texthack-plus\target\classes' 'com.team7.texthackplus.cli.CLI'
==================================
Current Corpus: english.txt | Domain Mode: OFF (General)
1. Select Corpus
2. Toggle Domain Mode
3. Exact Match (KMP Algorithm on Corpus / Text)
4. Fuzzy Match (Levenshtein Spell-Checker / Distance)
5. Document Similarity (Suffix Array + LCP Plagiarism Analysis)
6. Citation-Flow Analysis (Edmonds-Karp Max Flow)
7. Task Scheduling Demo (Priority Queue / MaxHeap)
8. Primality Test & Hash Sizing (Miller-Rabin)
9. Exit
Enter your choice: 7
--- Task Scheduling Engine (MaxHeap Priority Queue) ---
1. Default Predefined Job Set
2. Enter Custom Job Queue
Choice [1/2]: 1
Using predefined batch text processing jobs.
---- Scheduling Result ----
Execution Job Order: 2 3 1 4
Algorithm:           Priority Job Scheduler
Complexity:          O(n log n)
Time (ms):           8
Operations:          8
--------------------------
```

Figure 4.7: Task scheduling on the predefined job set: execution order 2 3 1 4 (priority job scheduler, O(n log n)).

```
PS C:\texthack-plus>  & 'C:\Users\prudh\.antigravity-ide\extensions\redhat.java-1.56.0-win32-x64\jre\21.0.12.1-win32-x86_64\bin\java.exe' '-XX:+ShowCodeDetailsInExceptionMessages' '-cp' 'C:\texthack-plus\target\classes' 'com.team7.texthackplus.cli.CLI'
==================================
Current Corpus: english.txt | Domain Mode: OFF (General)
1. Select Corpus
2. Toggle Domain Mode
3. Exact Match (KMP Algorithm on Corpus / Text)
4. Fuzzy Match (Levenshtein Spell-Checker / Distance)
5. Document Similarity (Suffix Array + LCP Plagiarism Analysis)
6. Citation-Flow Analysis (Edmonds-Karp Max Flow)
7. Task Scheduling Demo (Priority Queue / MaxHeap)
8. Primality Test & Hash Sizing (Miller-Rabin)
9. Exit
Enter your choice: 8
--- Primality Test & Hash Table Sizing (Miller-Rabin) ---
1. Test single number for primality
2. Compute Optimal Next Prime Capacity for CustomHashMap
Choice [1/2]: 2
Enter expected number of keys (e.g., 500): 600
---- Hash Table Capacity Recommendation ----
Expected Key Count:     600
Optimal Prime Capacity: 1201 buckets
Load Factor Target:     < 50% (uniform probing)
Algorithm:              Miller-Rabin Primality Test
Computation Time (ms):  77.340
-----------------------------------------
```

Figure 4.8: Primality test and hash sizing (Miller-Rabin): for 600 expected keys the recommended capacity is the prime 1201.

4.2 TEST 1: COMPONENT TEST HARNESSES

The repository contains nine assertion-based test harnesses. They were compiled together with the sources on JDK 21 and each was run with assertions enabled (`java -ea`). All nine passed; between them they contain 86 assertion statements.

Test class	Component checked	Assertions	Result
KMPMatcherTest	KMP: pattern found, pattern not found, empty pattern	6	Passed
LevenshteinTest	Edit distance values and early-stop behaviour	6	Passed

Test class	Component checked	Assertions	Result
SuffixTreeSimilarityTest	Suffix array and LCP on "banana#"; longest common substring	24	Passed
MaxFlowCitationTest	Maximum flow on test networks	2	Passed
JobSchedulerTest	Priority order, shorter job first on ties	5	Passed
MillerRabinPrimalityTest	Primes and composites	11	Passed
TokenizerTest	English and Hindi tokenisation	11	Passed
IngestionTest	DocumentStore add/retrieve; CorpusLoader loads all 3 corpora	7	Passed
QueryServiceTest	Dispatcher routing for the query types	14	Passed

Table 4.1: Test harness results.

4.3 TEST 2: KMP ON A WORST-CASE INPUT FOR NAIVE SEARCH

For this experiment the text consists only of the character "a" (length n) and the pattern is m - 1 characters "a" followed by "b", with m = n / 20. The pattern never occurs. A naive matcher must compare about (n - m + 1) x m characters, because it matches almost the whole pattern at every alignment. The table gives the number of character comparisons counted by the project's own KMP implementation, the analytically computed naive comparison count for the same input, and the measured KMP time (best of five runs after JVM warm-up).

Text length n	Pattern length m	KMP comparisons	Naive comparisons (analytic)	Ratio	KMP time (ms)
20,000	1,000	40,998	19,001,000	463x	0.223
40,000	2,000	81,998	76,002,000	927x	0.125
80,000	4,000	163,998	304,004,000	1854x	0.253
160,000	8,000	327,998	1,216,008,000	3707x	0.520

Table 4.2: KMP versus the naive comparison count on the worst-case family (measured on JDK 21).

KMP performs about 2n comparisons whatever the pattern, so its work doubles when n doubles, whereas the naive count grows roughly four-fold, consistent with O(n * m) when m is proportional to n. Wall-clock times at this scale are fractions of a millisecond and are affected by JIT compilation, so the operation counts are the more reliable evidence.

4.4 TEST 3: SCALING OF THE OTHER ALGORITHMS

Operation counts and times were also recorded on random inputs over the DNA alphabet {A, C, G, T}, using the project's own classes and counters.

Algorithm	Input size	Operations	Time (ms)	Observation
KMP (match at end of text)	n = 1,000	1,261	0.018	Operations about 1.25 n for a 20-character pattern
KMP	n = 10,000	12,522	0.195	
KMP	n = 100,000	124,809	0.819	
KMP	n = 500,000	625,071	3.390	Linear growth

Algorithm	Input size	Operations	Time (ms)	Observation
Levenshtein	n = m = 100	10,000	0.820	Operations = n x m exactly
Levenshtein	n = m = 500	250,000	8.335	
Levenshtein	n = m = 1,000	1,000,000	6.389	
Levenshtein	n = m = 2,000	4,000,000	24.331	Quadratic growth
Suffix array LCS	n = 100 per text	200	0.170	Scan makes 2n comparisons; time is dominated by sorting
Suffix array LCS	n = 500 per text	1,000	0.499	
Suffix array LCS	n = 1,000 per text	2,000	1.205	
Suffix array LCS	n = 2,000 per text	4,000	3.282	

Table 4.3: Measured operation counts and run times (best of several runs, JDK 21, single machine).

4.5 TEST 4: MILLER-RABIN PRIMALITY

Number	Expected	Result (k = 5)	Rounds executed
97	Prime	Prime	5
561 (Carmichael number)	Composite	Composite	1
7,919	Prime	Prime	5
1,000,000,007	Prime	Prime	5
1,000,000,009	Prime	Prime	5
2,147,483,647 (2^31 - 1)	Prime	Prime	5
1,000,000,011 (divisible by 3)	Composite	Composite	1
999,999,999,989	Prime	Prime	5

Table 4.4: Miller-Rabin on known primes and composites. Bases are drawn from a time-seeded generator, so the round at which a composite is rejected can vary between runs.

4.6 Observations

- All nine test harnesses pass, and the main CLI scenarios produce the expected answers (edit distance 3 for saturday/sunday, a single KMP match at index 4, maximum flow 23 on the predefined network, job order 2 3 1 4).
- KMP's comparison count stays near 2n even on the adversarial input, while the naive count grows with n x m.
- Levenshtein's operation count is exactly n x m, confirming the quadratic dynamic-programming cost; the two-row table keeps memory linear.
- The suffix array approach is simple and gives the exact longest common substring, but its quicksort over suffix comparisons is the bottleneck. A linear-time suffix array construction would improve it.
- Miller-Rabin is probabilistic: it is certain on composites it rejects, and very reliable with several rounds on primes.
- The whole engine uses only its own collections for data handling; java.io and java.nio are used solely to read corpus files.

4.7 CONCLUSION

TextHack+ integrates advanced algorithmic ideas into a working text-processing system. By implementing every data structure and algorithm by hand, without the java.util collections, the project shows both how these techniques work and what they cost. The layered architecture (corpus ingestion, algorithm engine, query service, user interface), the support for English, Hindi and medical corpora, and the uniform reporting of complexity, runtime and operation counts make the system a practical educational, benchmarking and research tool. Automated tests verify each component, and the measurements confirm the expected linear behaviour of KMP and the quadratic behaviour of edit distance.

4.8 FUTURE SCOPE

- **Parallel and distributed processing:** extend the custom data structures to multi-threading and distributed processing for much larger corpora.
- **Advanced visualisation:** step-by-step graphical views of max-flow citation analysis and suffix structure construction.
- **Domain and language expansion:** add modules for legal or scientific literature and improve tokenisation for complex Indian scripts.
- **Faster similarity:** replace the quicksort-based suffix array with a linear-time construction, and add all-matches output to the web exact-match endpoint.
- **Fuller web interface:** expose citation-flow and task-scheduling queries in the web UI alongside the existing four tabs.

REFERENCES

[1] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, *Introduction to Algorithms*, 3rd ed., MIT Press, 2009.

[2] D. E. Knuth, J. H. Morris and V. R. Pratt, "Fast pattern matching in strings," *SIAM Journal on Computing*, vol. 6, no. 2, pp. 323-350, 1977.

[3] V. I. Levenshtein, "Binary codes capable of correcting deletions, insertions and reversals," *Soviet Physics Doklady*, vol. 10, no. 8, pp. 707-710, 1966.

[4] U. Manber and G. Myers, "Suffix arrays: a new method for on-line string searches," *SIAM Journal on Computing*, vol. 22, no. 5, pp. 935-948, 1993.

[5] T. Kasai, G. Lee, H. Arimura, S. Arikawa and K. Park, "Linear-time longest-common-prefix computation in suffix arrays and its applications," in *Proc. CPM*, 2001.

[6] J. Edmonds and R. M. Karp, "Theoretical improvements in algorithmic efficiency for network flow problems," *Journal of the ACM*, vol. 19, no. 2, pp. 248-264, 1972.

[7] M. O. Rabin, "Probabilistic algorithm for testing primality," *Journal of Number Theory*, vol. 12, no. 1, pp. 128-138, 1980.

[8] D. Gusfield, *Algorithms on Strings, Trees, and Sequences*, Cambridge University Press, 1997.

[9] Project source code: https://github.com/prudhvisaikari-hub/DSA3-PROJECT

[10] Oracle, *Java Platform, Standard Edition Documentation*; Spring Boot Reference Documentation (spring.io).